S.No: 1	Exp. Name: ***Registration Form using Table***	**Date: 2026-08-18**

Aim:
Build a registration form laid out inside an HTML table.

Form Fields

- **Name**: A text input with `name="name"`.
- **Password**: A password input with `name="password"`.
- **E-mail**: An email input with `name="email"`.
- **Phone Number**: A telephone input with `type="tel"` and `name="phone"`.
- **Sex**: Two radio buttons for "Male" and "Female", both with `name="sex"`.
- **Date of Birth**: Three dropdowns with `name="day"`, `name="month"`, and `name="year"`.
- **Languages Known**: Four checkboxes for English, Telugu, Hindi, and Tamil, each with a matching lowercase `name` attribute.
- **Address**: A textarea with `name="address"`.

Table Layout

- Place each label in one column and its input in the adjacent column.
- Set the table border to 1.

Buttons

- A <button> element with `type="submit"` and the text "Submit".
- A <button> element with `type="reset"` and the text "Reset".

The form should look like this.

Registration Form

Name:	
Password:	
E-mail:	
Phone Number:	
Sex:	○ Male ○ Female
Date of Birth:	Day Month Year
Languages Known:	☐ English ☐ Telugu ☐ Hindi ☐ Tamil
Address:	
Submit Reset	

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Registration Form</title>
</head>
<body>
<h2>Registration Form</h2>
<form>
        <table border="1">
                <tr>
                        <td><label>name</label></td>
                        <td><input type ="text" name="name"></td>
                </tr>
                <tr>
                        <td><label>password</label></td>
                        <td><input type="password"
name="password"></td>
                </tr>
                <tr>
                        <td><label>E-mail</label></td>
                        <td><input type="email" name ="email"
</td>
                </tr>
                <tr>
                        <td><label>phone number</label></td>
                        <td><input type="tel" name="phone"><td>
                </tr>
                <tr>
                        <td><label>sex</label></td>
                        <td>
                                <input type="radio" name="sex"
value="male">male
                                <input type="radio" name="sex"
value="female">female
                        </td>
                </tr>
                <tr>
                        <td><label>date of birth</label></td>
                        <td>
                                <select name="day"
                                        <option>day</option>
```

```
                                        <option>1</option>
                                        <option>2</option>
                                        <option>3</option>
                                        <option>4</option>
                                        <option>5</option>
                                </select>
                                <select name="month">
                                        <option>month</option>
                                        <option>jan</option>
                                        <option>feb</option>
                                        <option>march</option>
                                        <option>april</option>
                                        <option>may</option>
                                        <option>june</option>
                                </select>
                                <select name="year">
                                        <option>year</option>
                                        <option>2004</option>
                                        <option>2005</option>
                                        <option>2006</option>
                                        <option>2007</option>
                                </select>
                        </td>
                </tr>
                <tr>
                        <td><label>languages known</label></td>
                        <td>
                                <input type="checkbox"
name="english">english
                                <input type="checkbox"
name="telugu">telugu
                                <input type="checkbox"
name="hindi">hindi
                                <input type="checkbox"
name="tamil">tamil
                        </td>
                </tr>
                <tr>
                        <td><label>address</label></td>
                        <td>
                                <textarea name="address" rows="4"
cols="30"></textarea>
                        </td>
                </tr>
                <tr>
                        <td colspan="2">
```

```
                                <button type="submit">
submit</button>
                                <button
type="reset">reset</button>
                            </td>
                </tr>
        </table>
</form>
</body>
</html>
```

S.No: 2	Exp. Name: ***Ravi's Portfolio Page***	**Date: 2026-09-08**

Aim:

Ravi is designing his personal portfolio webpage. He wants each section to look visually distinct. Style the text in each section as described below.

- In the <header>, the <h1> should have `font-family: Verdana`, `color: royalblue`, `text-shadow: 3px 3px 8px gray`, and `text-align: center`.
- In the <main>, the <h2> should have `text-transform: uppercase`, `color: darkred`, and `letter-spacing: 2px`.
- In the <main>, the <p> should have `font-size: 16px`, `color: #333`, and `line-height: 1.7`.
- In the <footer>, the <p> should have `text-align: center`, `font-style: italic`, and `color: darkgreen`.

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Ravi’s Portfolio Page</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header>
    <h1>Welcome to My Portfolio</h1>
  </header>

  <main>
    <h2>about me</h2>
    <p>Hello! I'm Ravi, a passionate front-end developer who
loves designing simple and elegant web experiences for users.</p>
  </main>

  <footer>
    <p>© 2025 Ravi Sharma | Designed with creativity and code.
</p>
  </footer>
</body>
</html>
```

styles.css

```
/*Write your code here*/
header h1{
        font-family : verdana;color : royalblue;
        text-shadow : 3px 3px 8px gray;
        text-align  : center;
}
main h2{
        text-transform : uppercase;
        color : darkred;
        letter-spacing : 2px;
}
main p{
        font-size:16px;
        color:#333;
        line-height:1.7;
}
footer p{
        text-align:center;
        font-style:italic;
        color:darkgreen;
}
```

S.No: 3	Exp. Name: ***Simple Calculator***	**Date: 2026-09-08**

Aim:

You are provided with an HTML file that defines the structure of a basic calculator. The calculator supports the following arithmetic operations:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)

Your task is to complete the JavaScript file (script.js) so that the calculator functions correctly.

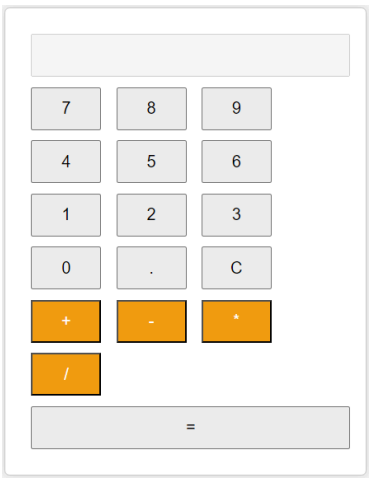
Provided Files

HTML File (index.html)

The HTML file already includes:

- A display area to show the current input or result.
- Buttons for digits (0–9).
- A decimal point button (.).
- Operation buttons (+, −, *, /).
- An equals button (=).
- A clear button (C).

You do not need to modify the HTML file.



Complete the JavaScript Functionality:

You must complete the logic in **script.js** by implementing the following functions correctly:

1. appendNumber(number)

This function is called when a number button is clicked.
It should:

- Allow multi-digit number input.
- Append the clicked digit to the current input.
- Handle decimal input properly (prevent multiple decimals in a single number).

2. setOperation(operation)

This function is called when an arithmetic operator (+, −, *, /) is clicked.
It should:

- Store the current input as the previous operand.
- Store the selected operation.
- Prepare the calculator to receive the next number.
- Prevent setting an operation if there is no current input.

3. calculate()

This function is called when the equals (=) button is clicked.
It should:

- Perform the selected arithmetic operation using the stored previous value and current value.
- Display the result.
- Handle division by zero by displaying:

```
Cannot divide by zero
```

If either the previous value or current value is empty, display:

```
Error
```

4. clearDisplay()

This function is called when the clear (C) button is clicked.
It should:

- Reset the current input.
- Reset the previous input.
- Reset the selected operation.
- Clear the display.

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Simple Calculator</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            height: 100vh;
            background-color: #f4f4f4;
        }
        .calculator {
            border: 1px solid #ccc;
            padding: 20px;
            border-radius: 5px;
            background-color: #fff;
            box-shadow: 0 0 10px rgba(0,0,0,0.1);
        }
        .calculator input, .calculator button {
            margin: 5px;
            padding: 10px;
            font-size: 16px;
        }
        .calculator input {
            width: 100%;
            box-sizing: border-box;
        }
        .calculator button {
            width: 22%;
        }
        .calculator button.operator {
            background-color: #f39c12;
            color: #fff;
        }
    </style>
</head>
<body>
    <div class="calculator">
        <input type="text" id="display" disabled>
```

```
        <br>
        <button onclick="appendNumber('7')">7</button>
        <button onclick="appendNumber('8')">8</button>
        <button onclick="appendNumber('9')">9</button>
        <br>
        <button onclick="appendNumber('4')">4</button>
        <button onclick="appendNumber('5')">5</button>
        <button onclick="appendNumber('6')">6</button>
        <br>
        <button onclick="appendNumber('1')">1</button>
        <button onclick="appendNumber('2')">2</button>
        <button onclick="appendNumber('3')">3</button>
        <br>
        <button onclick="appendNumber('0')">0</button>
        <button onclick="appendNumber('.')">.</button>
        <button onclick="clearDisplay()">C</button>
        <button class="operator" onclick="setOperation('+')">+
</button>
        <button class="operator" onclick="setOperation('-')">-
</button>
        <button class="operator" onclick="setOperation('*')">*
</button>
        <button class="operator"
onclick="setOperation('/')">/</button>
        <br>
        <button style="width: 100%;" onclick="calculate()">=
</button>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

script.js

```
let currentInput = '';
let previousInput = '';
let operation = null;

function appendNumber(number) {
        if (number === '.' && currentInput.includes('.'))
                return;
        currentInput += number;
        document.getElementById("display").value=currentInput;
}

function setOperation(op) {
    if (currentInput === ''){
                return;
        }
        previousInput = currentInput;
        operation = op;
        currentInput = '';
}

function clearDisplay() {
    currentInput ='';
        previousInput ='';
        operation = null;
   document.getElementById("display").value='';
}

function calculate() {

    if (previousInput === '' || currentInput === '' ){
                document.getElementById("display").value =
'Error';
                return;
           }
        let previous = parseFloat(previousInput);
        let current = parseFloat(currentInput);
        let result;
        if(operation === '+'){
                result=previous + current;
        }
        else if(operation === '-'){
                result=previous-current;
        }
        else if(operation === '*'){
```

```
                result=previous*current;
        }
        else if(operation ==='/'){
                if (current === 0){

document.getElementById("display").value="Cannot divide by zero";
                        return;
                }
                result=previous/current;
        }
        currentInput=result.toString();
        previousInput='';
        operation='';
        document.getElementById("display").value=currentInput;
}

function updateDisplay() {
    document.getElementById('display').value = currentInput;
}

function printToDisplay(message) {
    document.getElementById('display').value = message;
}
```

S.No: 4	Exp. Name: ***Dynamic To-Do List with Modals***	**Date: 2026-09-28**

Aim:

You have joined a front-end team building a productivity app. The HTML and CSS for a To-Do List app are already provided. Write the JavaScript in `script.js` to make the app interactive.

Your Task

1. Add Task

When the user types a task and clicks the Add button, add a new `<li>` to the list with `id="task-list"`. Each item must include:

- A `<span>` holding the task text
- Two buttons labelled `Edit` and `Delete`

2. Edit Task with Modal

Clicking `Edit` on a task should:

- Open `#edit-modal` by setting its `display` to `block`
- Show the current task text inside `#edit-input`
- Update the task text when the user clicks `Save`

The modal should close when:

- The user clicks `Save`
- The user clicks the close icon at the top right
- The user clicks outside the modal

3. Delete Task with Confirmation

Clicking `Delete` on a task should:

- Open `#delete-modal` by setting its `display` to `block`
- Remove the task if the user clicks `Yes`
- Close the modal without deleting if the user clicks No
- Close the modal if the user clicks outside the confirmation box

Note:

- Trigger the **Add** action when the user clicks the **Add** button. You may implement this using either the button's $click$ event or the form's submit event (using `event.preventDefault()` if needed).

Source Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Dynamic To-Do List</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="container">
    <h1>To-Do List</h1>
    <form id="task-form">
      <input type="text" id="task-input" placeholder="Add a new
task" required />
      <button type="submit" id="add-task">Add</button>
    </form>
    <ul id="task-list"></ul>
  </div>

  <!-- Edit Modal -->
  <div id="edit-modal" class="modal">
    <div class="modal-content">
      <span class="close-btn" id="close-edit-modal">&times;
</span>
      <h2>Edit Task</h2>
      <input type="text" id="edit-input" />
      <button id="save-edit">Save</button>
    </div>
  </div>

  <!-- Delete Confirmation Modal -->
  <div id="delete-modal" class="modal">
    <div class="modal-content">
      <p>Are you sure you want to delete this task?</p>
      <button id="confirm-delete">Yes</button>
      <button id="cancel-delete">No</button>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
// Write your code here

const task=document.getElementById('task-form');
const taskip=document.getElementById('task-input');
const taskList=document.getElementById('task-list');
const edit=document.getElementById('edit-modal');
const editip=document.getElementById('edit-input');
const del=document.getElementById('delete-modal');
const addButton=document.getElementById('add-btn');
const saveEdit=document.getElementById('save-edit');
const closeEdit=document.getElementById('close-edit-modal');
const deletemodal=document.getElementById('delete-modal');
const confirmdelete=document.getElementById('confirm-delete');
const canceldelete=document.getElementById('cancel-delete');

let activeTask=null;

task.addEventListener('submit',function (e) {
        e.preventDefault();
        const taskTest=taskip.value.trim();
        if(taskTest==="") return;
        const li=document.createElement('li');
        const span=document.createElement('span');
        span.textContent=taskTest;
        li.appendChild(span);
        const editBtn=document.createElement('button');
        editBtn.textContent='Edit';
        editBtn.className='edit-btn';
        li.appendChild(editBtn);
        const deleteBtn = document.createElement('button');
        deleteBtn.textContent = 'Delete';
        deleteBtn.className = 'delete-btn';
        li.appendChild(deleteBtn);
        taskList.appendChild(li);
        taskip.value="";

});

taskList.addEventListener('click',function (e) {
        if(e.target.classList.contains('edit-btn')){
                activeTask = e.target.closest('li');
                const currentText =
activeTask.querySelector('span').textContent;
                editip.value = currentText;
```

```
                edit.style.display='block';
        }
        else if(e.target.classList.contains('delete-btn')){
                activeTask = e.target.closest('li');
                del.style.display='block';
        }
});

saveEdit.addEventListener('click',function(){
        const updText=editip.value.trim();
        if(activeTask && updText !== ""){

activeTask.querySelector('span').textContent=updText;
        }
        edit.style.display='none';
});

confirmdelete.addEventListener('click',function(){
        if(activeTask){
                activeTask.remove();
        }
        del.style.display='none';
});

canceldelete.addEventListener('click',function(){
        del.style.display='none';
});

closeEdit.addEventListener('click',function(){
        edit.style.display='none';
});

window.addEventListener('click',function(e){
        if (e.target===edit){
                edit.style.display = 'none';
        }
        if (e.target===del){
                del.style.display = 'none';
        }
});
```

S.No: 5	Exp. Name: ***Node.js Event Logger Using Packages, Events, Timers, and File I/O***	**Date: 2026-09-28**

Aim:

In this experiment, you will build a Node.js event logging system that connects four core concepts into one working flow. The `server.js`, browser UI, and CSS file are already provided in the default code. Your only task is to implement two things inside `logger.js`.

Your Task

Implement the following inside `logger.js`:

- Create a class **LogEmitter** that extends the built-in `EventEmitter` class.
- Implement the **startLogging** function. It must do the following:
  - Create an instance of **LogEmitter**.
- Register a listener on the `"log"` event. Each time the event fires, the listener must build a log entry in this exact format and append it to `LOG_FILE` using `fs.appendFileSync`:
- `[<uuid>] <timestamp> — <message>\n`
- where `uuid` comes from `uuidv4()` and `timestamp` comes from new `Date().toISOString()`.Use `setInterval` to emit the `"log"` event every 2000 ms with the message `"Event fired"`.Use `clearInterval` to stop the interval after 5 ticks.Emit a `"done"` event on the emitter after the 5th tick.Return the **LogEmitter** instance.

Note: Click **Start** in the browser UI to trigger your implementation and observe the live dashboard. Click **Reset** to clear and run again.

Source Code:

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES 2024-2028-CSM-C ID: A24126552156 Page No: 19

my-app/logger.js

```
const EventEmitter = require("events");
const fs = require("fs");
const path = require("path");
const { v4: uuidv4 } = require("uuid");

const LOG_FILE = path.join(__dirname, "logs", "events.log");

if (!fs.existsSync(path.join(__dirname, "logs"))) {
  fs.mkdirSync(path.join(__dirname, "logs"));
}

// ─── TASK 1: Create a custom EventEmitter
──────────────────────────────────────────

// Create a class LogEmitter that extends EventEmitter.
// Write your code here
class LogEmitter extends EventEmitter{

}

// ─── TASK 2: Implement the startLogging function
──────────────────────────────────

// This function must:
//   1. Create an instance of LogEmitter
//   2. Register a listener on the "log" event that:
//        a. Builds a log entry in this exact format:
//           [<uuid>] <timestamp> — <message>\n
//           where uuid comes from uuidv4() and timestamp from
new Date().toISOString()
//        b. Appends the entry to LOG_FILE using
fs.appendFileSync
//   3. Use setInterval to emit the "log" event every 2000ms with
the message "Event fired"
//   4. Use clearInterval to stop the interval after 5 ticks
//   5. Emit a "done" event on the emitter after the 5th tick
//   6. Return the LogEmitter instance
function startLogging() {
  // Write your code here
        const emitter = new LogEmitter();
        emitter.on("log", (message) => {
                const uuid = uuidv4();
                const timestamp = new Date().toISOString();
                const logEntry = `[${uuid}] ${timestamp} —
${message}\n`;
                fs.appendFileSync(LOG_FILE,logEntry);
```

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES 2024-2028-CSM-C ID: A24126552156 Page No: 20

```
        });
        let count = 0;
        const interval = setInterval(()=>{
                count++;
                emitter.emit("log", "Event fired");
                if (count === 5){
                        clearInterval(timer);
                        emitter.emit("done");

                }
        },2000);
        return emitter;
}
module.exports = { startLogging, LOG_FILE };
```

my-app/package.json

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "Node.js Events, Timers, Packages and File I/O",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "uuid": "^9.0.0"
  }
}
```

my-app/public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Node.js Event Logger</title>
  <link rel="stylesheet" href="/style.css" />
</head>
<body>

  <div class="page-header">
    <h1>Node.js Event Logger</h1>
    <p class="subtitle">Monitor live event activity powered by
your Node.js implementation</p>
  </div>

  <div class="main">

    <div class="stat-grid">
      <div class="stat-card">
        <div class="stat-label">Status</div>
        <div class="stat-value" id="stat-status">
          <span class="badge idle">Idle</span>
        </div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Ticks Fired</div>
        <div class="stat-value big" id="stat-ticks">0 <span
class="of">/ 5</span></div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Last Event Time</div>
        <div class="stat-value mono" id="stat-time">—</div>
      </div>
      <div class="stat-card">
        <div class="stat-label">Log File Size</div>
        <div class="stat-value mono" id="stat-size">0 bytes</div>
      </div>
    </div>

    <div class="card">
      <div class="card-title">Last Generated UUID</div>
      <div class="uuid-display" id="stat-uuid">—</div>
```

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES 2024-2028-CSM-C ID: A24126552156 Page No: 22

```
    </div>

    <div class="card">
      <div class="card-title">Event Log</div>
      <div class="empty-state" id="empty-state">
        <span class="empty-icon">○</span>
        <p>No events yet. Click Start to begin.</p>
      </div>
      <div class="log-feed" id="log-feed"></div>
    </div>

    <div class="btn-row">
      <button id="start-btn"
onclick="handleStart()">Start</button>
      <button id="reset-btn" class="btn-secondary"
onclick="handleReset()">Reset</button>
    </div>

  </div>

  <script>
    let pollInterval = null;
    let lastCount = 0;

    function setStatus(type, label) {
      document.getElementById("stat-status").innerHTML = '<span
class="badge ' + type + '">' + label + '</span>';
    }

    window.handleStart = async function () {
      document.getElementById("start-btn").disabled = true;
      document.getElementById("reset-btn").disabled = true;
      setStatus("running", "Running");
      try {
        const res = await fetch("/start", { method: "POST" });
        const data = await res.json();
        if (data.status === "error") {
          setStatus("error", "Error");
          document.getElementById("stat-uuid").textContent =
data.message;
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
          return;
        }
        document.getElementById("empty-state").style.display =
"none";
```

```
        lastCount = 0;
        pollInterval = setInterval(pollLogs, 600);
      } catch (err) {
        setStatus("error", "Error");
        document.getElementById("start-btn").disabled = false;
        document.getElementById("reset-btn").disabled = false;
      }
    };

    window.handleReset = async function () {
      clearInterval(pollInterval);
      await fetch("/reset", { method: "POST" });
      lastCount = 0;
      document.getElementById("stat-ticks").innerHTML = '0 <span
class="of">/ 5</span>';
      document.getElementById("stat-time").innerHTML = "—";
      document.getElementById("stat-size").textContent = "0
bytes";
      document.getElementById("stat-uuid").innerHTML = "—";
      document.getElementById("log-feed").innerHTML = "";
      document.getElementById("empty-state").style.display =
"flex";
      document.getElementById("start-btn").disabled = false;
      document.getElementById("reset-btn").disabled = false;
      setStatus("idle", "Idle");
    };

    async function pollLogs() {
      try {
        const res = await fetch("/logs");
        const data = await res.json();
        const lines = data.lines || [];

        if (lines.length !== lastCount) {
          lastCount = lines.length;
          document.getElementById("stat-ticks").innerHTML =
lines.length + ' <span class="of">/ 5</span>';
          document.getElementById("stat-size").textContent =
data.fileSize + " bytes";

          if (lines.length > 0) {
            const lastLine = lines[lines.length - 1];
            const uuidMatch = lastLine.match(/\[([^\]]+)\]/);
            const timeMatch = lastLine.match(/\]\s+([^\s]+)\s+/);
            if (uuidMatch) document.getElementById("stat-
uuid").textContent = uuidMatch[1];
```

```
            if (timeMatch) document.getElementById("stat-
time").textContent = timeMatch[1].replace("T", " ").replace("Z",
" UTC");
          }

          const feed = document.getElementById("log-feed");
          feed.innerHTML = lines.map(function(line, i) {
            const uuidMatch = line.match(/\[([^\]]+)\]/);
            const timeMatch = line.match(/\]\s+([^\s]+)\s+/);
            const msgMatch  = line.match(/\u2014\s+(.*)/);
            const uuid = uuidMatch ? uuidMatch[1] : "—";
            const time = timeMatch ? timeMatch[1].replace("T", "
").replace("Z", " UTC") : "—";
            const msg  = msgMatch  ? msgMatch[1] : line;
            return '<div class="log-row">'
              + '<span class="log-num">#' + (i + 1) + '</span>'
              + '<div class="log-details">'
              + '<span class="log-msg">' + msg + '</span>'
              + '<span class="log-meta">' + time + '</span>'
              + '<span class="log-uuid">' + uuid + '</span>'
              + '</div></div>';
          }).join("");
          feed.scrollTop = feed.scrollHeight;
        }

        if (!data.isLogging && lines.length >= 5) {
          clearInterval(pollInterval);
          setStatus("done", "Done");
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
        }

        if (!data.isLogging && lines.length === 0) {
          clearInterval(pollInterval);
          setStatus("error", "Error");
          document.getElementById("start-btn").disabled = false;
          document.getElementById("reset-btn").disabled = false;
        }

      } catch (err) {
        clearInterval(pollInterval);
      }
    }
  </script>
```

```
</body>
</html>
```

my-app/public/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: "Segoe UI", sans-serif;
  background: #f0f2f5;
  min-height: 100vh;
  color: #1a202c;
}

.page-header {
  background: #fff;
  border-bottom: 1px solid #e2e8f0;
  padding: 24px 40px;
  text-align: center;
}

.page-header h1 {
  font-size: 1.4rem;
  font-weight: 700;
  color: #1a202c;
  margin-bottom: 4px;
}

.subtitle { font-size: 0.83rem; color: #718096; }

.main {
  max-width: 680px;
  margin: 32px auto;
  padding: 0 24px;
  display: flex;
  flex-direction: column;
  gap: 20px;
}

.stat-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
}

.stat-card {
  background: #fff;
  border: 1px solid #e2e8f0;
  border-radius: 12px;
```

```
  padding: 18px 20px;
}

.stat-label {
  font-size: 0.72rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.07em;
  color: #a0aec0;
  margin-bottom: 10px;
}

.stat-value { font-size: 0.95rem; font-weight: 600; color:
#2d3748; }

.stat-value.big {
  font-size: 1.8rem;
  font-weight: 700;
  color: #1a202c;
  line-height: 1;
}

.stat-value.mono {
  font-family: monospace;
  font-size: 0.82rem;
  color: #4a5568;
  word-break: break-all;
}

.of { font-size: 1rem; color: #a0aec0; font-weight: 500; }

.badge {
  display: inline-block;
  padding: 3px 10px;
  border-radius: 20px;
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.badge.idle    { background: #edf2f7; color: #718096; }
.badge.running { background: #ebf8ff; color: #2b6cb0; border: 1px
solid #90cdf4; }
.badge.done    { background: #f0fff4; color: #276749; border: 1px
solid #9ae6b4; }
```

```
.badge.error   { background: #fff5f5; color: #c53030; border: 1px
solid #feb2b2; }

.card {
  background: #fff;
  border-radius: 12px;
  border: 1px solid #e2e8f0;
  padding: 20px 24px;
}

.card-title {
  font-size: 0.72rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.07em;
  color: #a0aec0;
  margin-bottom: 12px;
}

.uuid-display {
  font-family: monospace;
  font-size: 0.88rem;
  color: #3182ce;
  font-weight: 600;
  background: #ebf8ff;
  border-radius: 7px;
  padding: 10px 14px;
  word-break: break-all;
  letter-spacing: 0.03em;
}

.empty-state {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 10px;
  padding: 32px 20px;
  text-align: center;
}

.empty-icon { font-size: 2rem; color: #e2e8f0; line-height: 1; }
.empty-state p { font-size: 0.83rem; color: #a0aec0; }

.log-feed {
  display: flex;
```

```
  flex-direction: column;
  gap: 8px;
  max-height: 280px;
  overflow-y: auto;
}

.log-row {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  background: #f7fafc;
  border-radius: 8px;
  padding: 10px 14px;
  border-left: 3px solid #4299e1;
}

.log-num {
  font-size: 0.72rem;
  font-weight: 700;
  color: #a0aec0;
  min-width: 24px;
  padding-top: 2px;
}

.log-details {
  display: flex;
  flex-direction: column;
  gap: 3px;
  flex: 1;
}

.log-msg { font-size: 0.88rem; font-weight: 600; color: #2d3748;
}
.log-meta { font-size: 0.75rem; font-family: monospace; color:
#718096; }
.log-uuid { font-size: 0.72rem; font-family: monospace; color:
#a0aec0; word-break: break-all; }

.btn-row { display: flex; gap: 12px; }

button {
  flex: 1;
  padding: 11px;
  background: #3182ce;
  color: #fff;
  border: none;
```

```
  border-radius: 8px;
  font-size: 0.9rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s, opacity 0.2s;
}

button:hover   { background: #2b6cb0; }
button:disabled { opacity: 0.5; cursor: not-allowed; }

.btn-secondary {
  background: #fff;
  color: #4a5568;
  border: 1.5px solid #e2e8f0;
}

.btn-secondary:hover { background: #f7fafc; }
```

my-app/server.js

```
const http = require("http");
const fs = require("fs");
const path = require("path");

const PORT = 3000;

const SERVER_FILE = path.join(__dirname, "logger.js");
let lastMtime = fs.statSync(SERVER_FILE).mtimeMs;
setInterval(() => {
  try {
    const mtime = fs.statSync(SERVER_FILE).mtimeMs;
    if (mtime !== lastMtime) {
      console.log("logger.js changed — restarting...");
      process.exit(1);
    }
  } catch (e) {}
}, 1000);

function serveStatic(res, filePath, contentType) {
  fs.readFile(filePath, (err, data) => {
    if (err) {
      res.writeHead(404, { "Content-Type": "text/plain" });
      res.end("404 Not Found");
      return;
    }
    res.writeHead(200, { "Content-Type": contentType });
    res.end(data);
  });
}

let isLogging = false;

function handleRequest(req, res) {
  if (req.method === "GET" && req.url === "/") {
    serveStatic(res, path.join(__dirname, "public",
"index.html"), "text/html");
    return;
  }

  if (req.method === "GET" && req.url === "/style.css") {
    serveStatic(res, path.join(__dirname, "public", "style.css"),
"text/css");
    return;
  }
```

```
  if (req.method === "POST" && req.url === "/start") {
    if (isLogging) {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "already running" }));
      return;
    }
    try {
      delete require.cache[require.resolve("./logger")];
      const { startLogging, LOG_FILE } = require("./logger");
      if (typeof startLogging !== "function") {
        res.writeHead(500, { "Content-Type": "application/json"
});
        res.end(JSON.stringify({ status: "error", message:
"startLogging is not implemented yet." }));
        return;
      }
      const logDir = path.join(__dirname, "logs");
      if (!fs.existsSync(logDir)) fs.mkdirSync(logDir);
      fs.writeFileSync(LOG_FILE, "");
      isLogging = true;
      const emitter = startLogging();
      emitter.on("done", () => { isLogging = false; });
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "started" }));
    } catch (err) {
      isLogging = false;
      res.writeHead(500, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ status: "error", message:
err.message }));
    }
    return;
  }

  if (req.method === "POST" && req.url === "/reset") {
    isLogging = false;
    try {
      delete require.cache[require.resolve("./logger")];
      const { LOG_FILE } = require("./logger");
      if (fs.existsSync(LOG_FILE)) fs.writeFileSync(LOG_FILE,
"");
    } catch (e) {}
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ status: "reset" }));
    return;
  }
```

```
  if (req.method === "GET" && req.url === "/logs") {
    try {
      delete require.cache[require.resolve("./logger")];
      const { LOG_FILE } = require("./logger");
      const content = fs.existsSync(LOG_FILE) ?
fs.readFileSync(LOG_FILE, "utf8") : "";
      const lines = content.trim().split("\n").filter(Boolean);
      const fileSize = fs.existsSync(LOG_FILE) ?
fs.statSync(LOG_FILE).size : 0;
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ lines, isLogging, fileSize }));
    } catch (err) {
      res.writeHead(200, { "Content-Type": "application/json" });
      res.end(JSON.stringify({ lines: [], isLogging: false,
fileSize: 0 }));
    }
    return;
  }

  res.writeHead(404, { "Content-Type": "text/plain" });
  res.end("Not Found");
}

const server = http.createServer(handleRequest);
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

S.No: 6	Exp. Name: ***Developing a Single Page Application using RESTful APIs***	**Date: 2026-10-05**

Aim:

You are developing a Movie Management System for an online streaming platform. The application allows users to add new movies, update movie ratings, delete movies, and view all available movies.

The application behaves as a Single Page Application (SPA). Instead of reloading the page after every operation, it communicates with the Express.js server using RESTful APIs through JavaScript's `fetch()` method.

The MongoDB connection, Movie model, project configuration, HTML, CSS, and the basic JavaScript functions have already been provided. Your task is to complete the missing code in both **index.js** and **script.js** so that the application works correctly.

Your Tasks

Complete the missing code in **index.js** by implementing the following RESTful API routes.

1. Create Movie

Create a POST route at `/add-movie` that:

- Accepts book details ($title$, $director$, $year$, and $rating$) in JSON format.
- Stores the received data as a new **Movie** document in the database.

Responds with:

```
Movie added: <movie-title>
```

2.View Movies

Create a GET route at `/movies` that:

- Retrieves all movies from the database using `Movie.find()`.
- Returns the retrieved documents as JSON.
- If an error occurs, respond with status code 500 and an appropriate error message.

3. Update Movie rating

Create a PUT route at `/update-movie` that:

- Accepts $title$ and $newRating$ from the request body.
- Updates only the rating of the matching movie using the $set operator.
- If the update is successful, respond with:

```
Movie '<title>' rating updated successfully
```

- If no matching movie is found, respond with:

```
No movie found with title '<title>'
```

- If an error occurs, respond with status code 500.

4. Delete Movie

Create a DELETE route at `/movies/:title` that:

- Deletes the movie whose title matches the route parameter.
- If the movie is deleted successfully, respond with: {"message": "Movie deleted successfully"}

- If no movie exists respond with: {"message": "Movie not found"}
- If an error occurs, respond with status code 500 and an appropriate error message.

Complete the missing code in **script.js** by completing `fetch()` requests for the following functions.

- **addMovie()** → Write a `fetch()` request that sends a POST request to `/add-movie` with the provided movie details in JSON format.
- **updateMovie()** → Write a `fetch()` request that sends a PUT request to `/update-movie` with the provided title and newRating in JSON format.
- **deleteMovie()** → Write a `fetch()` request that sends a DELETE request to `/movies/:title` by appending the movie title from the input field, encoded using `encodeURIComponent()`, to `/movies/`.
- **viewMovies()** → Write a `fetch()` request that sends a GET request to `/movies` and retrieves the movie list for display.

Run the application and verify that movies can be added, deleted, updated, and retrieved successfully.

Note: Do not modify the provided application setup. Write only the missing code in **index.js** and **script.js**.

Source Code:

express-app/index.js

```
const express = require('express');
const mongoose = require('mongoose');
const path = require('path');
const Movie = require('./models/movieModel');

const app = express();
app.use(express.json());
app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

let dbName = '';
let dbHost = '';
let dbPort = '';

mongoose.connect('mongodb://mongo:27017/libraryDB', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => {
  console.log('Database connected successfully!');

  const db = mongoose.connection;
  dbName = db.name;
  dbHost = db.host;
  dbPort = db.port;
})
.catch((err) => {
  console.error('Database connection failed:', err.message);
});

app.get('/ui', (req, res) => {
  res.render('index');
});

// Write POST /add-movie
app.post('/add-movie',async (req,res)=>{
        try{
                const {title, director, year, rating}=req.body;
                const newMovie=new
Movie({title,director,year,rating});
                await newMovie.save();
                res.send(`Movie added: ${title}`);
        }catch (err){
```

```
                res.status(500).send(err.message);
        }
});


// Write GET /movies
app.get('/movies',async (req,res)=>{
        try{
                const movies=await Movie.find({});
                res.json(movies);
        }catch(err) {
                res.status(500).json({error:err.message});
        }
});



// Write PUT /update-movie
app.put('/update-movie',async (req,res)=>{
        try{
                const { title, newRating }=req.body;
                const result=await Movie.updateOne(
                        {title:title},
                        {$set : {rating: newRating }}
                );
                if(result.matchedCount > 0){
                         res.send(`Movie '${title}' rating
updated successfully`);
                }
                else{
                        res.send(`No Movie found with title
'${title}'`);
                }
        }catch (err){
                res.status(500).send(err.message);
        }
});


// Write DELETE /movies/:title
app.delete('/movies/:title',async (req,res)=>{
        try{
                const title=req.params.title;
                const result=await
Movie.deleteOne({title:title});
                if (result.deletedCount > 0){
```

```
                        res.json({"message":"Movie deleted
successfully"});
                }
                else{
                        res.json({"message":"Movie not found"});
                }
        }catch (err){
                res.status(500).json({error:err.message});
        }
});



// -------------------- DATABASE DETAILS --------------------

app.get('/', (req, res) => {
  res.json({
    dbName,
    dbHost,
    dbPort
  });
});


// -------------------- SERVER --------------------

app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

express-app/public/script.js

```
function hideAll() {
    document.querySelectorAll(".page").forEach(page =>
page.style.display = "none");
}

function showPage(id) {
    hideAll();
    document.getElementById(id).style.display = "block";
    document.getElementById("output").innerHTML = "";
}

function goHome() {
    hideAll();
    document.getElementById("home").style.display = "block";
    document.getElementById("output").innerHTML = "";
}



// write the fetch for adding movie
async function addMovie(){
        const response=await fetch('/add-movie',{
                method:'POST',
                headers:{
                        'Content-Type':'application/json'
                },


        body: JSON.stringify({
            title: document.getElementById("title").value,
            director: document.getElementById("director").value,
            year: Number(document.getElementById("year").value),
            rating:
Number(document.getElementById("rating").value)
        })
        });

    document.getElementById("output").innerHTML = await
response.text();
}


// write the fetch for update
async function updateMovie(){
```

```
        const response=await fetch('/update-movie',{
                method:'PUT',
                headers:{
                        'Content-Type':'application/json'
                },


        body: JSON.stringify({
          title: document.getElementById("updateTitle").value,

newRating:Number(document.getElementById("newRating").value)
      })
        });

    document.getElementById("output").innerHTML = await
response.text();
}

// Write your code here for Delete Movie

async function deleteMovie() {
        const title=document.getElementById('deleteTitle').value;
        const response=await
fetch(`/movies/${encodeURIComponent(title)}`,{
                method:'DELETE'
        });

    document.getElementById("output").innerHTML = await
response.text();
}

async function viewMovies() {
    hideAll();
    document.getElementById("viewPage").style.display = "block";
// write the fetch to view
        const response=await fetch('/movies');
        const movies=await response.json();
    let table = `
    <table>
        <tr>
            <th>Title</th>
            <th>Director</th>
            <th>Year</th>
            <th>Rating</th>
        </tr>
    `;
```

```
    movies.forEach(movie => {

        table += `
        <tr>
            <td>${movie.title}</td>
            <td>${movie.director}</td>
            <td>${movie.year}</td>
            <td>${movie.rating}</td>
        </tr>
        `;

    });

    table += "</table>";
    document.getElementById("movieTable").innerHTML = table;
}
```

S.No: 7	Exp. Name: ***Creating and Nesting Function and Class Components***	**Date: 2026-09-22**

Aim:

You are developing a React application for a digital library. The application should display information about a book by creating and nesting React components. In this activity, the React application setup has already been provided. Your task is to create the **Book** function component and the **App** class component and nest the **Book** component inside the **App** component.

Your Task

1. Inside **Book.jsx**, create a function component named **Book** that returns a `<div>` with the class **book-card**, containing:
    - An <h2> element with the text "The Great Gatsby".
    - A paragraph element with the text "Author: F. Scott Fitzgerald".

Export the **Book** component.

2. In the **App.jsx** file:
    - Create a class component named **App** that extends **Component**.
    - Import the Book component from `./Book`.
    - Inside the `render()` method, display an <h3> element with the text "Books".
    - Create a `<div>` with the class name **library**, and nest the `<Book />` component inside it.
    - Return the JSX from the `render()` method.
    - Export the **App** component as the default export.

Expected Output:

When the application is executed, it should display:

```
Books
--------------------------------
The Great Gatsby
Author: F. Scott Fitzgerald
--------------------------------
```

where the **Book** component is nested inside the **App** component.

Note:

- Both **App.jsx** and **Book.jsx** are in the same path.
- Initially, the page will show a placeholder until the required code is completed.

Source Code:

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES | 2024-2028-CSM-C | ID: A24126552156 | Page No: 43

functionAndClassComponents/src/App.jsx

```
import { Component } from "react";
import Book from "./Book";
import ErrorBoundary from "./components/ErrorBoundary"

// Create the App class component here...
class App extends Component{
        render(){
                return(
                        <ErrorBoundary>
                                <h3>Books</h3>
                                <div className="library"><Book />
</div>
                        </ErrorBoundary>
                );
        }
}
export default App;
```

functionAndClassComponents/src/Book.jsx

```
// Create the Book function component here...
function Book(){
        return(
                <div className="book-card">
                <h2>{"The Great Gatsby"}</h2>
                        <p>{"Author: F. Scott Fitzgerald"}</p>
                </div>
        );
}
export default Book;
```

S.No: 8	Exp. Name: ***Employee Management using JSON Server (GET and POST Methods)***	**Date: 2026-10-06**

Aim:

You are building an **Employee Management** tool that allows a manager to add, view, update, and delete employee records.

The data is served by a JSON Server running at `http://localhost:3001/employees`. Your React application must communicate with this server using the **Fetch API** over four HTTP methods: **GET**, **POST**, **PUT**, and **DELETE**.

Your Task

In EmployeeForm.jsx

Build a form with the following input fields:

- Employee Name
- Department
- Designation
- Salary

Include a submit button labelled `Add Employee`. When the button is clicked, validate that all fields are filled. If adding a new employee, send the data using a **POST** request. If editing an existing employee, send the updated data using a **PUT** request. On success, clear all input fields and refresh the employee table.

In EmployeeTable.jsx

On page load, fetch all employee records from the server using a **GET** request and display them in a table with the following columns:

- ID
- Name
- Department
- Designation
- Salary
- Actions

Each row must have an **Edit** button that populates the form with that employee's existing data, and a **Delete** button that sends a **DELETE** request to remove that employee and immediately refreshes the table.

Source Code:

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES | 2024-2028-CSM-C | ID: A24126552156 | Page No: 45

employee-management/jsonserver/db.json

```
{
  "employees": [
    { "id": 1, "name": "Amit", "department": "Production",
"designation": "Engineer", "salary": 45000 },
    { "id": 2, "name": "Priya", "department": "HR",
"designation": "Manager", "salary": 38000 },
    { "id": 3, "name": "Rahul", "department": "Sales",
"designation": "Executive", "salary": 42000 }
  ]
}
```

employee-management/package.json

```
{
  "name": "employee-management",
  "private": true,
  "version": "1.0.0",
  "scripts": {
    "dev": "vite"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^5.0.0",
    "vite": "^7.0.0"
  }
}
```

employee-management/src/App.jsx

```
import { useEffect, useState } from "react";
import EmployeeForm from "./EmployeeForm";
import EmployeeTable from "./EmployeeTable";

function App() {
  const [employees, setEmployees] = useState([]);
  const [editEmployee, setEditEmployee] = useState(null);

  const loadEmployees = async () => {
    const response = await
fetch("http://localhost:3001/employees");
    const data = await response.json();
    setEmployees(data);
  };

  useEffect(() => {
    loadEmployees();
  }, []);

  return (
    <div className="page">
      <div className="topbar">
        <div className="topbar-icon">👥</div>
        <div>
          <h1>Employee Management</h1>
          <p>Manage your team records in one place</p>
        </div>
      </div>
      <EmployeeForm
        loadEmployees={loadEmployees}
        editEmployee={editEmployee}
        setEditEmployee={setEditEmployee}
      />
      <EmployeeTable
        employees={employees}
        loadEmployees={loadEmployees}
        setEditEmployee={setEditEmployee}
      />
    </div>
  );
}

export default App;
```

employee-management/src/EmployeeForm.jsx

```
import { useState, useEffect } from "react";

function EmployeeForm({ loadEmployees, editEmployee,
setEditEmployee }) {
  const [name, setName] = useState("");
  const [department, setDepartment] = useState("");
  const [designation, setDesignation] = useState("");
  const [salary, setSalary] = useState("");

  useEffect(() => {
    if (editEmployee) {
      setName(editEmployee.name);
      setDepartment(editEmployee.department);
      setDesignation(editEmployee.designation);
      setSalary(editEmployee.salary);
    } else {
      setName("");
      setDepartment("");
      setDesignation("");
      setSalary("");
    }
  }, [editEmployee]);

  const handleSubmit = async (e) => {
    // Validate all fields are filled
    // If editEmployee exists, send PUT request to update
    // Otherwise send POST request to add
    // Clear all fields and call loadEmployees after success
          e.preventDefault();
if(!name.trim()||!department.trim()||!designation.trim()||!salary)
{
        alert("All fields are required!");
        return;
}
const employeeData=
{name,department,designation,salary:Number(salary)};
          try{
                  if(editEmployee){
                          const response=await
fetch(`https://localhost:3001/employees/${editEmployee.id}`,{
                                  method:"PUT",
                                  headers:{"Content-
Type":"application/json"},body:JSON.stringify(employeeData)
                          });
```

```
                        if(response.ok){
                                await response.json();
                                setName("");
                                setDepartment("");
                                setSalary("");
                                setEditEmployee(null);
                                loadEmployees();
                        }
                }
                else{
                      const response=await
fetch(`http://localhost:3001/employees`,{
                                method:"POST",
                                headers:{"Content-
Type":"application/json"},body:JSON.stringify(employeeData)
                        });
                        if(response.ok){
                                await response.json();
                                setName("");
                                setDepartment("");
                                setDesignation("");
                                setSalary("");
                                loadEmployees();

                        }
                }
          }
          catch(error){
                  console.log("Employee details are not
saved:",error);
          }
  };

  return (
    <div className="card">
      <div className="card-header">
        {editEmployee ? "Update employee" : "Add new employee"}
      </div>
      <div className="form-grid">
        <div className="field">
          <label>Employee name</label>
          <input
            type="text"
            className="emp-name"
            placeholder="e.g. Sneha Kulkarni"
            value={name}
```

```
            onChange={(e) => setName(e.target.value)}
          />
        </div>
        <div className="field">
          <label>Department</label>
          <input
            type="text"
            className="emp-department"
            placeholder="e.g. Engineering"
            value={department}
            onChange={(e) => setDepartment(e.target.value)}
          />
        </div>
        <div className="field">
          <label>Designation</label>
          <input
            type="text"
            className="emp-designation"
            placeholder="e.g. Senior Developer"
            value={designation}
            onChange={(e) => setDesignation(e.target.value)}
          />
        </div>
        <div className="field">
          <label>Salary</label>
          <input
            type="number"
            className="emp-salary"
            placeholder="e.g. 55000"
            value={salary}
            onChange={(e) => setSalary(e.target.value)}
          />
        </div>
      </div>
      <div className="form-actions">
        {editEmployee && (
          <button className="btn" onClick={() =>
setEditEmployee(null)}>
            Cancel
          </button>
        )}
        <button className="btn btn-primary" onClick=
{handleSubmit}>
          {editEmployee ? "Update employee" : "Add employee"}
        </button>
      </div>
```

```
    </div>
  );
}

export default EmployeeForm;
```

employee-management/src/EmployeeTable.jsx

```
function EmployeeTable({ employees, loadEmployees,
setEditEmployee }) {
  const handleDelete = async (id) => {
    // Send DELETE request to http://localhost:3001/employees/:id
    // Call loadEmployees to refresh the table
          try{
                  const response=await
fetch(`http://localhost:3001/employees/${id}`,{
                          method:"DELETE"
                  });
                  if(response.ok){
                          loadEmployees();
                 }
        }
          catch(error){
                  console.error("Error deleting emp",error);
          }

  };

  return (
    <div className="card">
      <div className="card-header">Employee records</div>
      <div className="table-wrap">
        <table className="employee-table">
          <thead>
            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Department</th>
              <th>Designation</th>
              <th>Salary</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody className="employee-body">
                          {employees && employees.length>0?(
                employees.map((emp)=>(
                        <tr key={emp.id}>
                                <td>{emp.id}</td>
                                <td>{emp.name}</td>
                                <td>{emp.department}</td>
                                <td>{emp.designation}</td>
                                <td>{emp.salary}</td>
```

```
                                <td>
                                        <button
                                                type="button"
                                                className="btn
btn-sm"
                                                onClick=
{()=>setEditEmployee(emp)}

>Edit</button>button>
                                        <button type="button"
                                                className="btn
btn-sm btn-danger"
                                                onClick=
{()=>handleDelete(emp.id)}
                                                >Delete</button>
                                </td>
                        </tr>
                ))
                ):(
                <tr>
                        <td colSpan="6" style=
{{textAlign:"center"}}>No records found</td>
                </tr>
                )}
          </tbody>
        </table>
      </div>
    </div>
  );
}

export default EmployeeTable;
```

employee-management/src/ErrorBoundary.jsx

```
import React from "react";

class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }
  componentDidCatch(error, errorInfo) {
    console.error("Error in the Code:", error, errorInfo);
  }
  render() {
    if (this.state.hasError) {
      return (
        <div style={{ padding: "20px", color: "#333" }}>
          <h2>Something went wrong in the code.</h2>
          <pre>{this.state.error?.toString()}</pre>
        </div>
      );
    }
    return this.props.children;
  }
}

export default ErrorBoundary;
```

employee-management/src/index.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background: #f0f4f8;
  min-height: 100vh;
  color: #1a202c;
}

.page {
  max-width: 860px;
  margin: 0 auto;
  padding: 32px 24px;
}

.topbar {
  display: flex;
  align-items: center;
  gap: 12px;
  margin-bottom: 28px;
}

.topbar-icon {
  width: 42px;
  height: 42px;
  border-radius: 10px;
  background: #3b82f6;
  display: flex;
  align-items: center;
  justify-content: center;
  color: #ffffff;
  font-size: 20px;
}

.topbar h1 {
  font-size: 20px;
  font-weight: 600;
  color: #1a202c;
  margin: 0;
}
```

```
.topbar p {
  font-size: 13px;
  color: #718096;
  margin-top: 2px;
}

.stats {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 12px;
  margin-bottom: 20px;
}

.stat {
  background: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  padding: 14px 16px;
}

.stat-label {
  font-size: 11px;
  font-weight: 500;
  color: #a0aec0;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  margin-bottom: 4px;
}

.stat-value {
  font-size: 24px;
  font-weight: 600;
  color: #1a202c;
}

.card {
  background: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 12px;
  padding: 24px;
  margin-bottom: 20px;
}

.card-header {
  font-size: 12px;
```

```
  font-weight: 600;
  color: #718096;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  margin-bottom: 18px;
}

.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
  margin-bottom: 16px;
}

.field {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.field label {
  font-size: 12px;
  font-weight: 500;
  color: #4a5568;
}

.field input {
  height: 36px;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  background: #f7fafc;
  color: #1a202c;
  font-size: 14px;
  padding: 0 12px;
  outline: none;
  font-family: inherit;
  transition: border-color 0.15s, box-shadow 0.15s;
}

.field input:focus {
  border-color: #3b82f6;
  box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.12);
  background: #ffffff;
}

.field input::placeholder {
```

```
  color: #a0aec0;
}

.form-actions {
  display: flex;
  gap: 10px;
  justify-content: flex-end;
}

.btn {
  height: 36px;
  padding: 0 16px;
  border-radius: 8px;
  font-size: 13px;
  font-weight: 500;
  cursor: pointer;
  border: 1px solid #e2e8f0;
  background: transparent;
  color: #4a5568;
  font-family: inherit;
  display: inline-flex;
  align-items: center;
  gap: 6px;
  transition: background 0.15s;
}

.btn:hover {
  background: #f7fafc;
}

.btn-primary {
  background: #3b82f6;
  color: #ffffff;
  border-color: transparent;
}

.btn-primary:hover {
  background: #2563eb;
}

.btn-sm {
  height: 28px;
  padding: 0 10px;
  font-size: 12px;
  border-radius: 6px;
}
```

```
.btn-danger {
  color: #e53e3e;
  border-color: #fed7d7;
}

.btn-danger:hover {
  background: #fff5f5;
}

table {
  width: 100%;
  border-collapse: collapse;
  font-size: 13px;
}

thead tr {
  border-bottom: 1px solid #e2e8f0;
}

th {
  padding: 10px 14px;
  text-align: left;
  font-size: 11px;
  font-weight: 600;
  color: #a0aec0;
  text-transform: uppercase;
  letter-spacing: 0.06em;
}

td {
  padding: 13px 14px;
  color: #2d3748;
  border-bottom: 1px solid #f0f4f8;
}

tbody tr:last-child td {
  border-bottom: none;
}

tbody tr:hover {
  background: #f7fafc;
}

.id-cell {
  color: #a0aec0;
```

```
  font-size: 12px;
  font-family: monospace;
}

.salary-cell {
  font-weight: 600;
  font-family: monospace;
}

.badge {
  display: inline-flex;
  align-items: center;
  height: 22px;
  padding: 0 8px;
  border-radius: 6px;
  font-size: 11px;
  font-weight: 500;
}

.badge-dept {
  background: #ebf4ff;
  color: #2b6cb0;
}

.badge-desg {
  background: #faf5ff;
  color: #6b46c1;
}

.actions-cell {
  display: flex;
  gap: 6px;
  align-items: center;
}
```

employee-management/src/main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import ErrorBoundary from "./ErrorBoundary";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <ErrorBoundary>
    <App />
  </ErrorBoundary>
);
```

employee-management/vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
export default defineConfig({
  plugins: [react()],
  server: {
    port: 3000,
    host: true
  }
});
```

S.No: 9	Exp. Name: ***MongoDB CRUD Operations Using Mongoose***	**Date: 2026-10-05**

Aim:

You are developing a Book Management System for a library. The application stores book information in a MongoDB database using Mongoose. A Book model has already been provided. Your task is to complete the Express.js application by connecting to the database and implementing the required CRUD (Create, Read, Update, and Delete) operations.

Your Task

Open the **index.js** file and complete the missing code as described below:

1. Connect to MongoDB

Connect the application to the following MongoDB database:

`mongodb://mongo:27017/libraryDB`

Using the Mongoose connection object, store:

- the database name in `dbName`
- the database host in `dbHost`
- the database port in `dbPort`

2. Create Operation

Create a POST route at `/add-book` that:

- Accepts book details (`title`, `author`, `year`, and `price`) in JSON format.
- Stores the received data as a new **Book** document in the database.

Responds with:

`Book added: <book-title>`

3. Read Operation

Create a `GET` route at `/books` that:

- Retrieves all books from the database using `Book.find()`.
- Returns the retrieved documents as JSON.

If an error occurs, respond with status code 500 and an appropriate error message.

4. Update Operation

Create a PUT route at `/update-book` that:

- Accepts `title` and `newPrice` from the request body.
- Updates only the price of the matching book using the `$set` operator.

If the update is successful, respond with:

`Book '<title>' price updated successfully`

If no matching book is found, respond with:

`No book found with title '<title>'`

If an error occurs, respond with status code 500.

5. Delete Operation

Create a `DELETE` route at `/books/:title` that:

- Deletes the book whose title matches the route parameter using:
  `Book.deleteOne({ title: req.params.title })`

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES 2024-2028-CSM-C ID: A24126552156 Page No: 62

- If the book is deleted successfully, respond with: { "message": "Book deleted successfully" }
- If no matching book is found, respond with status code 404 and: { "message": "Book not found" }

If an error occurs, respond with status code 500 and an appropriate error message.

Run the application and verify that books can be added, retrieved, updated, and deleted successfully.

Note: The **Book** model and project configuration have already been provided. Write only the missing code without modifying the existing application setup.

Source Code:

express-app/index.js

```
const express = require('express');
const mongoose = require('mongoose');
const Book = require('./models/bookModel');

const app = express();

app.use(express.json());
app.set('view engine', 'ejs');
app.use(express.urlencoded({ extended: true }));

let dbName = 'libraryDB';
let dbHost = 'mongo';
let dbPort = '27017';

// Write your code here to connect to MongoDB
mongoose.connect(`mongodb://${dbHost}:${dbPort}/${dbName}`,{
        useNewUrlParser:true,
        useUnifiedTopology:true
});

app.get('/ui', (req, res) => {
  res.render('index');
});


// Write your code here for POST route (/add-book)
 app.post('/add-book',async(req,res)=>{
         try{
                 const{title,author,year,price}=req.body;
                 const newBook=new
Book({title,author,year,price});
                 await newBook.save();
                 res.status(200).send(`Book added: ${title}`);
         }catch(error){
                 res.status(500).send("Error adding book");
         }
 });

// Write your code here for GET route (/books)
app.get('/books',async(req,res)=>{
        try{
                const books=await Book.find({});
                res.status(200).json(books);
        } catch (error) {
```

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES 2024-2028-CSM-C ID: A24126552156 Page No: 64

```
                res.status(500).json({error:error.messsage});
        }
});

// Write your code here for PUT route (/update-book)
app.put('/update-book',async(req,res)=>{
        try{
                const{title,newPrice}=req.body;
                const result=await Book.updateOne({title:title},
{$set:{price:newPrice}});
                if (result.matchedCount===0){
                        return res.status(404).send(`No book
found with title'${title}'`);
                }
                res.status(200).send(`Book '${title}' price
updated successfully`);
        } catch (error) {
                res.status(500).send(error.message)
        }
});

// Write your code here for DELETE route (/books/:title)
app.delete('/books/:title',async(req,res)=>{
        try{
                const {title}=req.params;
                const result=await Book.deleteOne({title:title});
                if(result.deletedCount===0){
                        return
res.status(404).json({"message":"Book not found"});
                }
                res.status(200).json({"message":"Book deleted
successfully"});
        } catch (error) {
                res.status(500).json({"error":error.message});
        }
});

// -------------------- DATABASE DETAILS --------------------
app.get('/', (req, res) => {
  res.json({
    dbName,
    dbHost,
    dbPort
  });
});
```

```
// -------------------- SERVER --------------------
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

S.No: 10	Exp. Name: ***Weapon Assembly System***	**Date: 2026-10-05**

Aim:

You are working on the assembly interface for the **M97A Harbinger**, a weapon platform in a game studio's build pipeline. The rendering layer is already implemented and draws every component in a ghosted outline state until that component is installed. Your task is to complete the backend API routes in `backend/server.js` and the frontend API integration in `frontend/src/App.jsx` so the assembly system works end to end.

Backend — complete these routes in `backend/server.js`:

- Connect the Express application to MongoDB using the connection string provided in the `.env` file.
- Create a Mongoose schema and model for weapon parts with the fields `slot`, `label`, and `installed`.
- Implement a `GET /api/parts` route that retrieves all part records from MongoDB and returns them as JSON.
- Implement a `POST /api/parts/:id/install` route that sets the `installed` field of the matching part to `true` and returns the updated part.
- Implement a `POST /api/parts/reset` route that sets `installed` back to `false` for every part and returns the full updated list.

Frontend — complete `frontend/src/App.jsx`:

- Create a state variable to store the list of weapon parts.
- Use `useEffect()` to fetch all parts from the backend when the application loads.
- Implement the `handleInstall(id)` function to send a POST request that installs the selected part.
- Update the stored parts list with the part returned by the backend so the rendering layer redraws that component in its solid state.
- Implement the `handleReset()` function to send a POST request to the reset route and replace the stored parts list with the response.
- Pass the parts list to the `Weapon` component through the **parts** prop.

Expected Behavior

- On initial load every component of the weapon is drawn as a ghosted outline.
- Installing a component redraws it in solid metal with its energy cells lit.
- The fire control stays disabled until every component is installed.
- Firing the fully assembled weapon plays the muzzle flash, ejects casings, and recoils the platform.

Notes:

- The weapon rendering layer in **Weapon.jsx** has already been implemented.
- The MongoDB database is automatically seeded with the five weapon parts before the application starts.
- Do not modify the existing rendering layer or styling.

• Complete only the backend API implementation and the frontend API integration.

Source Code:

weaponAssembly/backend/server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
require("dotenv").config();

// Create the Express application
// Configure the cors and express.json middleware
const app=express();
app.use(cors());
app.use(express.json());

// Connect to MongoDB using the MONGO_URI environment variable
mongoose.connect(process.env.MONGO_URI)
        .then(()=>console.log("Connected to MongoDB
successfully"))
        .catch((err)=>console.log ("MongoDB connection
error:",err));

// Create a Part schema with the following fields:
const partSchema=new mongoose.Schema({

        slot:{type:String,required:true},
        label:{type:String,required:true},
        installed:{type:Boolean,default:false}
});
// - slot
// - label
// - installed

// Create the Part model
const Part=mongoose.model("Part",partSchema);

// Implement GET /api/parts
// Retrieve all parts from MongoDB
// Return them as JSON
app.get("/api/parts",async(req,res)=>{
        try{
                const parts=await Part.find({});
                res.json(parts);
        } catch (error) {
                res.status(500).json({error:"Failed to fetch
parts"});
        }
});
```

```
// Implement POST /api/parts/reset
// Set installed to false for every part
// Return the full updated list of parts
app.post("/api/parts/:id/install",async(req,res)=>{
        try {
                const updatedPart=await Part.findByIdndUpdate(
                        req.params.id,
                        {installed:true},{new:true}
                );
                if(!updatedPart){
                        return res.status(404).json({error:"part
not found"});
                }
                res.json(updatedPart);
        } catch (error) {
                res.status(505).json({error:"Failed to update
part"});
        }
});
// Implement POST /api/parts/:id/install
// Set installed to true for the part whose document ID matches
req.params.id
// Return the updated part
app.post("/api/parts/reset",async(req,res)=>{
        try {
                await Part.updateMany ({},{installed:false});
                const allParts=await Part.find({});
                res.json(allParts);
        } catch (error) {
                res.status(500).json({error:"Failed to reset"});
        }
});
// Start the server on the port specified in the .env file
const PORT=process.env.PORT || 5000;
app.listen(PORT,()=>{
        console.log('Server running on port ${PORT}');
});
```

weaponAssembly/frontend/src/App.jsx

```
import { useEffect, useState } from "react";
import Weapon from "./Weapon";

function App() {
  // Create parts state to store the list of weapon parts
        const [parts,setParts]=useState([]);

  const [firing, setFiring] = useState(false);

  // Fetch all parts from /api/parts when the app loads
        useEffect(()=>{
                fetch("/api/parts")
                  .then((res)=>res.json())
                  .then((data)=>setParts(data))
                  .catch((err)=>console.log("error:",err));
        },[]);

  // Implement handleInstall to POST to /api/parts/:id/install
and update that part in state
        const handleInstall=(id)=>{
                fetch(`/api/parts/${id}/install`,{
                        method:"POST"
                })
                .then((res)=>res.json())
                .then((updatedPart)=>{
                        setParts((prevParts)=>
                                prevParts.map((part)=>
(part._id===id?
updatedPart:part)));
                })
                  .catch((err)=>console.log("Error:",err));
        };
  // Implement handleReset to POST to /api/parts/reset and
replace the parts list
        const handleReset=()=>{
                fetch("/api/parts/reset",{
                        method:"POST"
                })
                  .then((res)=>res.json())
                  .then((data)=>setParts(data))
                  .catch((err)=>console.log("Error,err"));
        };
  const allInstalled = parts.length > 0 && parts.every((part) =>
part.installed);
```

```
  const handleFire = () => {
    const svg = document.getElementById("weapon");
    const flash = document.getElementById("muzzleFlash");
    if (!svg || !flash) return;
    setFiring(true);
    let i = 0;
    const shots = setInterval(() => {
      flash.innerHTML =
        '<ellipse cx="20" cy="127" rx="' + (20 + Math.random() *
12) +
        '" ry="' + (11 + Math.random() * 6) + '" fill="#EF9F27"
opacity="0.95"/>' +
        '<ellipse cx="16" cy="127" rx="' + (11 + Math.random() *
6) +
        '" ry="' + (6 + Math.random() * 3) + '"
fill="#FAC775"/>';
      flash.setAttribute("opacity", "1");
      for (let s = 0; s < 6; s++) {
        const spark =
document.createElementNS("http://www.w3.org/2000/svg", "circle");
        const ang = Math.PI + (Math.random() - 0.5) * 0.85;
        const dist = 40 + Math.random() * 55;
        spark.setAttribute("cx", "20");
        spark.setAttribute("cy", "127");
        spark.setAttribute("r", (1 + Math.random() *
1.8).toFixed(1));
        spark.setAttribute("fill", "#EF9F27");
        svg.appendChild(spark);
        spark.animate(
          [
            { transform: "translate(0,0)", opacity: 1 },
            { transform: "translate(" + Math.cos(ang) * dist +
"px, " + Math.sin(ang) * dist + "px)", opacity: 0 }
          ],
          { duration: 400, easing: "ease-out" }
        ).onfinish = () => spark.remove();
      }
      const casing =
document.createElementNS("http://www.w3.org/2000/svg", "rect");
      casing.setAttribute("x", "300");
      casing.setAttribute("y", "120");
      casing.setAttribute("width", "7");
      casing.setAttribute("height", "3");
      casing.setAttribute("rx", "1");
      casing.setAttribute("fill", "#BA7517");
```

```
      svg.appendChild(casing);
      casing.animate(
        [
          { transform: "translate(0,0) rotate(0)", opacity: 1 },
          { transform: "translate(" + (24 + Math.random() * 18) +
"px, " + (-26 - Math.random() * 14) + "px) rotate(180deg)",
opacity: 0.9 },
          { transform: "translate(" + (46 + Math.random() * 20) +
"px, 60px) rotate(420deg)", opacity: 0 }
        ],
        { duration: 750, easing: "ease-out" }
      ).onfinish = () => casing.remove();
      setTimeout(() => flash.setAttribute("opacity", "0"), 70);
      i++;
      if (i >= 10) { clearInterval(shots); setFiring(false); }
    }, 125);
  };

  return (
    <div className="app-wrapper">
      <div className="title-bar">
        <h1>M97A HARBINGER</h1>
        <p>ASSEMBLY TERMINAL / CLASS: TITAN</p>
      </div>
      <div className="weapon-stage">
        {/* Pass the parts list to the Weapon component through
the parts prop */}
        <Weapon parts={parts} firing={firing} />
      </div>
      <div className="part-list">
        {parts.map((part) => (
          <button
            key={part._id}
            className="part-btn"
            data-slot={part.slot}
            disabled={part.installed}
            onClick={() => handleInstall(part._id)}
          >
            {part.installed ? part.label + " installed" :
"Install " + part.label}
          </button>
        ))}
      </div>
      <div className="control-bar">
        <button className="btn-fire" disabled={!allInstalled}
onClick={handleFire}>
```

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES 2024-2028-CSM-C ID: A24126552156 Page No: 73

```
          FIRE
        </button>
        <button className="btn-reset" onClick={handleReset}>
          RESET
        </button>
      </div>
      <p className="status-line">
        {allInstalled ? "STATUS: ASSEMBLED" : "STATUS:
INCOMPLETE"}
      </p>
    </div>
  );
}

export default App;
```